

SemRec: A Real-Compiler-Grounded Divergence Oracle for Cross-Language Migration

($C \rightarrow \text{Rust}/\text{Go}/\text{Swift}/\text{C++}/\text{OCaml}/\text{Zig}/\text{WebAssembly}$ and $\text{Rust} \rightarrow C$) via σ -Bridge Product Programs

halley-labs

Abstract

Migrating C to a safety-checked language or runtime (Rust, Go, Swift, Zig, WebAssembly) demands tooling that compares *source-language* semantics rather than relying on runtime agreement or LLVM-level normalization. The central hazard is *undefined-behavior (UB) divergence*: a C input on which the program's result is not fixed by the standard (the optimizer may resolve it either way, or a sanitizer traps), while the safe-language port computes one deterministic, defined value. A single dynamic run cannot expose this gap. We present SEMREC, a repository-backed divergence oracle that encodes C and target semantics separately through a σ -bridge product program, finds a witnessing input or bounded execution with Z3, dataflow, or enumeration, and—crucially—*confirms every witness against real compilers* (`clang` under UndefinedBehaviorSanitizer, MemorySanitizer where available, `-Wunsequenced`, or a checked compiler-mode delta; `rustc`; `go`; and `zig`) so that no verdict ships unless an independent ground-truth harness confirms it. The oracle framework is a uniform plugin interface: each divergence class is an additive plugin, and a class automatically transfers across language pairs. The checked-in artifact implements a catalogue of 24 divergence classes—signed overflow, shift/division UB, `INT_MIN/-1`, out-of-bounds access, strict-aliasing (type-punning), sequence-point/unsequenced read-write UB, floating-point contraction and `-ffast-math` reassociation, VLA-bound, float $\rightarrow$ int overflow, pointer-provenance / address-space-overflow arithmetic, `restrict`-aliasing, overlapping `memcpy` vs. `memmove`-like slice copy, uninitialized struct-padding serialization vs. zero-padded safe serialization, `longjmp` to an exited VLA scope vs. structured unwinding, a scoped relaxed-atomics store-buffering ordering gap, an uninitialized-read/definedness analysis, two implementation-defined classes (bit-field packing and out-of-range enum representation), and a C/C++-boundary class (1 $\ll$ 31, undefined in C yet defined in C++20)—driven across seven target languages (Rust, Go, Swift, C++, a GC'd, exception-based OCaml port, and Zig `ReleaseSafe`, plus WebAssembly value-or-trap semantics under `wasmtime`) by a deterministic, byte-reproducible cross-pair regression matrix whose toolchain-backed cells are *all* confirmed against real compilers/runtimes (`clang`, `rustc`, `go`, `clang++`, `ocamlopt`, `zig`, `wasmtime`) plus an executable checked-libc contract build for preconditions UBSan does not cover and an MSan/auto-init padding confirmation for indeterminate object bytes. The framework is not limited to C-rooted undefined behaviour in the forward direction: a *Rust $\rightarrow$ C reverse pair* proves that a Rust-defined panic can become target-side C UB under FFI-style lowering, and a first *safe $\rightarrow$ safe* pair, Go $\rightarrow$ Rust—in which *neither* side has any undefined behaviour—catches the *defined-but-different* hazard where `INT_MIN/-1` is a defined modular value in Go yet a guaranteed panic in Rust, raising the matrix to **60 cells over 9 pairs**, with confirmation delegated to the real compiler/toolchain/runtime for every available cell. The new C $\rightarrow$ WebAssembly column contributes four runtime-backed integer cells: C UB vs. wasm wrapping, shift-mask, division-by-zero trap, and signed-division-overflow trap semantics. To make the breadth claim falsifiable, a second cross-pair generalization study replays the registry over the newly added pairs/directions and requires every available positive witness to confirm while every safe control remains unflagged; an executable per-pair soundness checker separately verifies the registry entry, confirmation mode, target semantics pack, positive witness, negative control, and live confirmation for C $\rightarrow$ Zig, C $\rightarrow$ C++20, Rust $\rightarrow$ C, C $\rightarrow$ WebAssembly, Go $\rightarrow$ Rust, and C $\rightarrow$ OCaml. Beyond pairwise checks, an *N-language consistency oracle* compiles one C source to ≥ 3 safe targets at once and flags the minority target whose live output disagrees, and a Lean typeclass SPI contract plus a target-pack conformance suite mechanically certify that every back end satisfies the plugin/pack obligations. For the integer-UB family we additionally ship an SMT-backed encoding whose equivalence to a human-readable specification is Z3-proved for *every* 32/64-bit input,

whose operational decision path agrees with an independent enumeration predicate on a deterministic 10,000-case differential harness, and which finds the rare overflow witnesses (e.g. `INT_MIN/-1`, one input in 2^{64}) in milliseconds where ordered enumeration exhausts its budget. For floating-point contraction, the artifact now retains the exact binary64 operand and result bit patterns and replays them through Z3's IEEE-754 FP theory before the real `clang/rustc` confirmation path runs. An optimization-level sweep additionally pins the exact `-0` level at which each no-sanitizer-traps UB class first changes the program's observable output. On a labelled divergence study (29 cases over 19 anchor classes) the oracle attains perfect precision and recall (tp = 23, fp = 0, fn = 0, tn = 528) where a 200,000-trial differential fuzzer finds nothing on the UB-invisible classes. A Tier-1 `c2rust`-output corpus separately runs the real `c2rust 0.22.1` translator on 12 real-library extraction units, byte-compares the regenerated machine output with the checked-in Rust, compiles the source and generated artifacts with `clang/rustc`, and checks the generated Rust artifacts against the verifier; a GitHub-mined `C→Rust` port sample ranks public port/rewrite repositories and byte-checks verifier-backed extraction units from three project families. A historical-CVE replay corpus indexes **60** NVD CVE ids across **8** CWE families and emits **120** `C→{Rust,Go}` minimized reproduction bundles: each bundle compiles a from-scratch weakness-class witness, proves the C/UBSan side traps on the CVE-class input, checks a safe negative input, and verifies that the target has one language-defined outcome. The corpus is explicitly scoped as weakness-class replay rather than vendored third-party vulnerable source. The same artifact also keeps the tool honest at semantic frontiers: volatile/inline-assembly/FFI/general atomic units cause loud abstention justified by real `clang IR`, while the scoped atomics oracle uses a deterministic model-level proof; a `ThreadSanitizer/go run -race/rustc` concurrency oracle proves that the same unsynchronized counter is C data-race UB, Go racy-but-defined, and rejected by Rust's safe `Send/Sync` discipline while synchronized repairs compile and run clean. We give a mechanized-friendly soundness development (σ -bridge coercion correctness, product-program isolation, a bounded-completeness boundary, Lean-checked class lemmas for strict-aliasing optimizer-exploitation and pointer-provenance trap-vs-defined witnesses, and an end-to-end theorem that counterexample packaging preserves the raw product-run witness), plus an independent Coq cross-check of the same core recorded-observable/product-witness theorem, a Lean-checked preservation theorem for the counterexample minimizer, and proof-carrying counterexamples: each confirmed source-UB witness serializes a canonical, hash-bound certificate over the recorded $P \wedge T \wedge C$ observation; the verdict-layer preimage is length-prefixed and Lean-proved injective, so canonicalization itself cannot merge distinct verdict payloads before SHA-256. An offline checker validates the certificate before a Lake-built checker re-validates the final `oracle_sound` inference. The artifact regenerates every headline number from content-hashed real-compiler runs, with an incremental cache whose warm hits are proved verdict-layer identical to cold compiler runs, compiler-backed artifacts that pin the exact `clang/rustc/go` version fingerprint and reject replay under a drifted toolchain, a deterministic process-pool harness whose merged hash is independent of worker scheduling, and a sharded-reproducibility proof whose per-shard canonical bytes repeat and whose concatenated shard verdict hash equals the unsharded run. A new flaky-toolchain detector reruns the same items and hashes the fuller compiler/runtime evidence layer, quarantining any item whose output evidence changes even if its coarse verdict does not; a distributed manifest pins per-shard item identities for cross-machine farming and rejects missing, duplicate, or drifted shard results before merging; and a result-store v2 schema separates the stable verdict hash from the environment-bound store hash so legacy artifacts migrate without weakening the reproducibility lemma. This sits alongside a supervised peak-RSS cap whose proof rejects resource exhaustion and requires the bounded verdict-layer hash to match the unbounded run, plus a cold-start CI gate that masks opportunistic toolchains while running the full non-compiler suite and then unmask them for one real `clang/UBSan+rustc` signed-overflow confirmation; on the checked artifact run this gate executes 1,420 non-compiler tests and the real compiler sample in 77.247 s, comfortably below the five-minute budget. The scale artifact includes a migration-scale corpus of **60,000** all-distinct `C→{Rust,Go}` programs totaling **1,044,000 LOC**: the census proves every program hash unique and the labels balanced within 1.33%, while a seeded live sample (12/12 agreement, hash `5ec6c935c7b0`) is replayed through real `clang/UBSan, rustc, and go`. A new soundness-regression gate enumerates all 60 registered oracle instances across 9 pairs, rejecting any plugin that lacks a per-pair soundness statement, matching confirmation mode, source-definedness premise, theorem/claim reference, and live witness probe; a generated soundness compendium maps every oracle instance to its theorem references and concrete witness unit.

1 Introduction

The software industry is undergoing a structural shift toward memory-safe languages. CISA's *Secure by Design* pledge [1] calls on software manufacturers to adopt memory-safe languages; the NSA has issued formal guidance recommending organizations transition from C/C++ to languages such as Rust [2]; and

the White House Office of the National Cyber Director has endorsed memory safety as a national security priority [3]. Major codebases are responding: the Chromium project reports that 70% of high-severity security bugs stem from memory safety violations [4], and Microsoft attributes a similar fraction to the same root cause [5]. The Linux kernel now accepts Rust modules [6], and Android has seen a measurable reduction in memory-safety vulnerabilities as the fraction of new code written in Rust has increased.

The invisible-bug problem. The most dangerous $C \rightarrow \text{Rust}$ translation bugs are *provably invisible* to every existing verification approach. Signed integer overflow is undefined behavior (UB) in C (C11 §6.5/5 [10]) but wraps modulo 2^w in Rust release builds; division of `INT_MIN` by `-1` is UB in C but wraps in Rust; shifts by $\geq w$ are UB in C but mask the shift amount in Rust. These divergences are *erased* when compiled to LLVM IR: the C compiler encodes “no signed wrap” as an `nsw` flag, while Rust omits it, but both produce the same `add i32` instruction. Moreover, on x86 hardware, C signed overflow *wraps identically* to Rust, so differential testing—even with edge-case-aware input generation—sees identical outputs and reports no divergence.

In our IR erasure experiment, 100% of UB divergences produced identical LLVM IR. In our differential testing experiment, UB-aware testing with 10,000 inputs *per pair* detected **0/30** UB divergences. IR-level tools (KLEE [11], SMACK [12], Alive2 [14]) also detect **0/30**. These bugs hide in every test suite, survive code review, and only surface when compiled with a different optimizer or on a different architecture.

Approach. SEMREC addresses this gap via σ -bridge product programs. Given a C function f_C and its Rust translation f_R , SEMREC:

1. Parses both independently (tree-sitter with recursive-descent fallback) and lowers to a shared typed SSA IR annotated with per-language overflow, shift, and division UB semantics.
2. Constructs a product program with shared symbolic inputs and disjoint variable namespaces, inserting σ -bridge coercion nodes at every program point where the semantic configurations σ_C and σ_R disagree.
3. Encodes the product program in QF_BV and discharges via Z3 [17]. C UB semantics become SMT assumptions (`assert_assume`); Rust wrapping semantics become concrete bitvector operations. UNSAT certifies bounded equivalence; SAT yields a counterexample; assumption violation yields a UB divergence witness.

Key result. On 30 IR-invisible UB divergences spanning 6 classes (overflow, shift, division, cast, negate, compound) plus 10 equivalent negative controls, SEMREC achieves **100% recall** (30/30 divergences detected), 93.8% precision, and 96.8% F1. Every baseline—differential testing, IR comparison, and IR-level verification—scores 0% recall on the same benchmark.

Contributions.

1. The **σ -bridge product program** construction for source-level cross-language equivalence verification with formally specified UB coercions at 26 divergence points (Section 2).
2. A **division UB encoding**: extending the σ -bridge to mark `SDIV/SREM` as UB for both division-by-zero and `INT_MIN/-1`, enabling detection of a class of bugs previously missed (Section 2).
3. **Theorem 3** (Soundness): UNSAT implies equivalence on all well-defined inputs within the loop bound (Appendix A).
4. The **UB-invisible benchmark**: 30 divergent + 10 equivalent pairs with ground truth, demonstrating 100% recall vs. 0% for all baselines (Section 4).
5. An **open-source tool** with a true-green test ratchet, targeted real-compiler regression tests, and a compiler-IR localizer that maps a reported C/Rust semantic split back to the exact `clang-AST` and `rustc-MIR` facts (Section 3).

Scope. SEMREC is a *bounded* verifier operating on individual function pairs. It excels at scalar UB verification (arithmetic, shift, division, casts). Struct/enum/iterator-heavy code has lower coverage due to parser and IR limitations, and we are forthright about these boundaries throughout the paper.

Venue framing. We frame the submission as a PLDI/OOPSLA/CAV-style semantics-and-systems tool paper: the novelty is the cross-language product-program account and sound-for-divergence evidence pipeline, while the C→Rust UB story remains the flagship empirical wedge that makes the general claim concrete.

2 Approach

This section presents the three pillars of SEMREC’s verification methodology: the σ -bridge product program construction, the divergence class taxonomy, and the soundness argument.

2.1 σ -Bridge Product Programs

The core idea is to execute both functions symbolically on shared inputs within a single SMT formula, inserting *coercion nodes* wherever the two languages’ semantics diverge.

Definition 1 (Semantic Configuration). *A semantic configuration is a function $\sigma : Op \times Type \rightarrow \{UB, WRAP, PANIC, SATURATE, CHECKED\}$ assigning to each arithmetic operation and type pair the language-mandated behavior on exceptional inputs. We write $\sigma_C = c11()$ for C11 semantics and $\sigma_R = rust_release()$ for Rust release-mode semantics. The implementation resides in `src/semantics/`.*

The five behaviors correspond to distinct SMT encodings in `src/smt/encoder.py`:

- **WRAP**: standard bitvector operation; modular arithmetic is the bitvector default.
- **UB**: bitvector operation plus a well-definedness guard added to the assumption set (`ctx.assert_assume`).
- **PANIC/TRAP**: bitvector operation plus a hard assertion (`ctx.assert_hard`) that fails if overflow occurs.
- **SATURATE**: result clamped to `[INT_MIN, INT_MAX]` via `z3.If`.
- **CHECKED**: overflow flag stored in a companion variable `{name}_overflow`.

Definition 2 (σ -Bridge Coercion). *A σ -bridge coercion at program point i is a triple $\sigma_i = (pre_i, post_i, dom_i)$ where $pre_i(\vec{x})$ is a precondition on operands, $post_i(\vec{x}, y_C, y_R)$ relates the C and Rust outputs, and dom_i specifies the applicable type domain. When pre_i holds, the coercion guarantees that both encodings produce semantically equivalent results.*

The `CoercionGenerator` class (`src/product_program/coercion.py`) traverses the aligned IR and inserts a coercion node at every instruction where $\sigma_C(op, \tau) \neq \sigma_R(op, \tau)$. Each coercion is generated by a specialized function (e.g., `generate_overflow_assertions`, `generate_division_assertions`) dispatched through a table mapping `DivergenceClass` to generator.

Product program construction. Algorithm 1 summarizes the construction, implemented in `ProductBuilder.build()` (`src/product_program/product.py`).

The `FunctionAligner` (`src/product_program/alignment.py`) performs a four-phase alignment: (1) greedy LCS-based block alignment with falthrough merging, (2) reorder detection for out-of-order blocks, (3) instruction-level Needleman–Wunsch alignment within matched blocks using cross-language similarity scoring (e.g., C `switch` vs. Rust `match`), and (4) structural similarity computation. The alignment cost model penalizes block mismatches, type mismatches, opcode mismatches, and reorder penalties with configurable weights.

Output assertion. The final formula is $\Phi = WellDefined(\sigma_C, f_C, \vec{x}) \wedge (ret_C(\vec{x}) \neq ret_R(\vec{x}))$. If Φ is UNSAT, no well-defined input produces differing outputs within the bound K . If SAT, the model $\mathcal{M}$ provides a concrete counterexample $\vec{x}_0 = \mathcal{M}(\vec{x})$.

SMT encoding. The encoder (`src/smt/encoder.py`) represents each w -bit integer as a `Z3 BitVec(w)`. Arithmetic maps to bitvector operations (`bvadd`, `bvsub`, `bvshl`, etc.). Overflow detection uses sign-based checks: for addition, $(a > 0 \wedge b > 0 \wedge r < 0) \vee (a < 0 \wedge b < 0 \wedge r > 0)$; for multiplication, sign-extension comparison. Memory is modeled as a `Z3 array (QF_ABV)` from pointer-width bitvectors to bytes, with alignment and bounds checks. The encoder generates separate contexts for C and Rust with distinct `SemanticConfig` instances via `encode_sigma_equivalence()`, producing shared variables, per-side return values, C-side well-definedness assumptions, and the divergence query $ret_C \neq ret_R$.

Algorithm 1 σ -Bridge Product Program Construction

```
1: procedure BUILDPRODUCT( $f_C, f_R, \sigma_C, \sigma_R, K$ )
2:    $\text{alignment} \leftarrow \text{FUNCTIONALIGNER.ALIGN}(f_C, f_R)$  ▷ Needleman–Wunsch on blocks
3:    $\vec{x} \leftarrow \text{CREATESHAREDINPUTS}(f_C.\text{params}, f_R.\text{params})$ 
4:   for each parameter pair  $(p_{C,i}, p_{R,i})$  do
5:      $\tau_i \leftarrow \max(\tau_{C,i}, \tau_{R,i})$  ▷ Join type: wider of the two
6:     Insert coerce( $x_i, \tau_{C,i}$ ) for C side
7:     Insert coerce( $x_i, \tau_{R,i}$ ) for Rust side
8:   end for
9:    $B_C \leftarrow \text{PREFIXRENAME}(f_C.\text{body}, \text{"c\_"})$ 
10:   $B_R \leftarrow \text{PREFIXRENAME}(f_R.\text{body}, \text{"r\_"})$ 
11:  Unroll all loops in  $B_C, B_R$  up to bound  $K$ 
12:  for each aligned instruction pair  $(I_C, I_R)$  do
13:    if  $\sigma_C(\text{op}(I_C), \text{type}(I_C)) \neq \sigma_R(\text{op}(I_R), \text{type}(I_R))$  then
14:      Insert  $\sigma$ -bridge coercion ( $\text{pre}, \text{post}, \text{dom}$ )
15:    end if
16:  end for
17:   $wd \leftarrow \text{WELLDEFINED}(\sigma_C, B_C, \vec{x})$ 
18:   $\Phi \leftarrow wd \wedge (\text{ret}_C(\vec{x}) \neq \text{ret}_R(\vec{x}))$ 
19:  return  $\Phi$ 
20: end procedure
```

2.2 Divergence Class Taxonomy

SEMPREC ships a catalogue of **24 divergence classes** (Table 1)—16 rooted in C undefined behaviour, four in C unspecified or indeterminate behaviour, three implementation-defined layout/representation or conversion classes, and one defined allowed-execution-set atomics class. Twenty-one of those catalogue entries are executable oracle rows today, and the C→Rust anchor exposes 20 registered oracle classes behind a single plugin contract (§3). The classification emerges from a systematic comparison of C17 [10] and target-language (Rust/Go/Swift/C++) semantics and is validated, class by class, against `src/ub_oracle/catalogue.py`. Every executable row is *anchored to a real witness*: the oracle’s verdict is not believed until the ground-truth harness re-runs the witness against actual compilers (§2.3, §4).

Cross-pair generality. Because each class shares one `divergence_class` key across language pairs, a single solver/dataflow/model witness drives the class against *every* target that implements it. The deterministic cross-pair regression matrix (§4) covers **9 language pairs** (C→Rust, C→Go, C→Swift, C→OCaml, C→C++, C→WebAssembly, C→Zig, and the *safe→safe* Go→Rust plus the reverse Rust→C pair) with 20, 18, 6, 4, 1, 4, 5, 1, and 1 classes respectively—**60 cells in total**—and is byte-reproducible per fresh process. When the corresponding host toolchain is present, each attempted cell must confirm against real `clang/rustc/go/clang++/ocaml/opt/zig/wasmtime` or the checked C contract build for pre-conditions UBSan does not cover. A second registry-backed generalization study (`generalization.run_generalization`) exercises the newer pairs and directions directly: one live positive witness and one safe control per available pair, including C→C++20, C→OCaml, Rust→C, and Go→Rust on our host, must respectively confirm and remain unflagged. This is a breadth/precision transfer claim—not a claim that semantically different pairs share one identical invariant. The companion checker (`pair_soundness.confirm_pair_soundness`) makes the pair statements machine-readable by checking the oracle registry, expected confirmation mode, target-pack class resolution, symbolic positive/negative controls, and live confirmation whenever the compiler/runtime is installed. The Rust→C pair reverses the usual polarity: the source is defined (a Rust panic) while the C target is UBSan-confirmed UB. The Go→Rust pair is qualitatively new in a different direction: *neither* source nor target has any undefined behaviour, so the framework’s reach extends past C-UB to both reverse-UB and *defined-but-different* regimes (§4.11). The C→C++ pair is qualitatively distinct: its witness, the byte-identical token `1 < 31`, is *undefined* in C (C17 6.5.7p4) yet *defined* in C++20 (which mandates two’s-complement and a modular shift, `[expr.shift]/2`)—so the divergence surface is the C/C++ language boundary itself, not a re-translation. The catalogue additionally records classes whose soundness frontier is *honest abstention* rather than a value witness (unspecified evaluation order with side effects, use-after-free and null-dereference as memory-shape hazards), documented as the boundary of the value-level oracle rather than silently asserted equivalent.

Table 1: The executable divergence-class catalogue. Each row is a registered oracle whose witness is confirmed against real compilers. “Mode” is the ground-truth confirmation discipline: TRAP = `clang -fsanitize` traps on a defined input while the target is defined; OPT = the same C source produces different output under two conforming compilations (no sanitizer can trap it) while the target is deterministic; CONTRACT = a real C build with an executable checked precondition traps; STATIC = real `clang -Wunsequenced` proves source UB while the target is defined; DEF = both the C and target programs are language-defined and deterministic, yet their observable output differs (an implementation-defined choice the port silently changes); MODEL = real compiled model checkers confirm a bounded allowed-execution-set gap; PAD = MSan (when available) or `clang -ftrivial-auto-var-init={pattern,zero}` proves a whole-struct digest depends specifically on padding bytes, while a zero-padding control and target serialization are deterministic.

Class	C (C17)	Safe target	Mode
Signed overflow	UB	wrap / panic	TRAP
Shift $\geq$ width	UB	defined shift	TRAP
Division by zero	UB	panic	TRAP
INT_MIN / -1	UB	panic	TRAP
Array out-of-bounds	UB	bounds-check panic	TRAP
Unsequenced read/write	UB	fixed eval order	STATIC
Float $\rightarrow$ int overflow	UB	saturating / panic	TRAP
VLA bound ≤ 0	UB	checked-len panic	TRAP
Pointer provenance / ovf.	UB	checked index	TRAP
Overlapping <code>memcpy</code>	UB	memmove-like slice copy	CONTRACT
Stale <code>longjmp</code> /VLA target	UB	structured unwind cleanup	CONTRACT
Uninitialized read	UB	zero/default-init	OPT
Uninitialized padding bytes	unspec.	zero-padded serialization	PAD
Strict aliasing (pun)	UB	defined value	OPT
<code>restrict</code> violation	UB	unique-borrow / GC	OPT
FP contraction (FMA)	unspec.	IEEE-strict	OPT
<code>-ffast-math</code> reassoc.	unspec.	IEEE-strict	OPT
Bit-field layout / packing	impl.-def.	unpacked fields	DEF
Out-of-range <code>enum</code>	impl.-def.	default variant	DEF
$1 \ll 31$ (C $\rightarrow$ C++)	UB	C++20-defined	TRAP
Relaxed atomics store-buffering	defined	SeqCst forbids all-zero	MODEL

Frontiers and concurrency. Not every semantic mismatch should produce a value witness. The implementation therefore contains two evidence-backed “do not guess” mechanisms. First, a foreign-effect detector rejects units containing `volatile`, inline `asm`, unknown `extern` calls, general atomics, signals, or non-local jumps; each abstention is checked against real `clang` IR (for example, four `load volatile` operations survive where the pure loop collapses to one). The exception is the deliberately scoped `atomic_litmus` oracle: it does not claim to observe a flaky runtime interleaving, but model-checks the bounded store-buffering execution set and compiles real C/Rust/Go atomics snippets that confirm C relaxed allows the all-zero outcome while SeqCst targets forbid it. Second, the concurrency oracle compiles and runs a small pattern catalogue under ThreadSanitizer and Go’s race detector, and compiles the corresponding Rust safe translation. The unsynchronized counter is called a race only because TSan and `go run -race` fire on real binaries; the Rust version is rejected by real `rustc`, while the `Mutex`, `atomic`, and `read-only` repairs compile, run, and stay clean under the available detectors. These paths are deliberately separate from the value-level SMT oracles: they prevent false equivalence claims at language features whose semantics include external effects or thread interleavings.

2.3 Soundness

We state the main soundness theorem, corresponding to `verify_soundness_theorem_1()` in `src/product_program/sound`

Theorem 3 (Soundness). *Let f_C be a C function and f_R its Rust translation. Let K be the loop unrolling bound. If the product program formula*

$$\Phi = \text{WellDefined}(\sigma_C, f_C, \vec{x}) \wedge ([f_C]_{\sigma_C}(\vec{x}) \neq [f_R]_{\sigma_R}(\vec{x}))$$

is unsatisfiable, then for all inputs $\vec{x}$ such that (i) f_C has defined behavior under C11 semantics, and

(ii) all execution paths through $f_C(\vec{x})$ and $f_R(\vec{x})$ have length $\leq K$, we have $\llbracket f_C \rrbracket(\vec{x}) = \llbracket f_R \rrbracket(\vec{x})$.

Proof sketch. Suppose Φ is UNSAT and, for contradiction, there exists $\vec{x}_0$ satisfying both conditions with $\llbracket f_C \rrbracket(\vec{x}_0) \neq \llbracket f_R \rrbracket(\vec{x}_0)$. By bitvector encoding faithfulness (Lemma 6, Appendix A), the Z3 encoding evaluates to the same values as the mathematical semantics. By well-definedness completeness (Lemma 7), the SMT predicate is true iff f_C avoids UB. By σ -bridge correctness (Lemma 8), coercions preserve output relationships at divergence points. By variable isolation (Lemma 9), the two sides share only the inputs. Together, $\vec{x}_0$ satisfies Φ , contradicting UNSAT. $\square$

Theorem 4 (Counterexample Soundness). *If Φ is satisfiable with model $\mathcal{M}$, then $\vec{x}_0 = \mathcal{M}(\vec{x})$ is a genuine divergence witness: $f_C(\vec{x}_0)$ has defined behavior under C11 and $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$.*

Theorem 5 (Completeness Characterization). *The equivalence verdict is complete (no false negatives) under four conditions:*

C1: All execution paths have length $\leq K$.

C2: All divergence-inducing operations belong to the handled σ -bridge classes.

C3: IR lowering succeeds without semantic information loss.

C4: The QF_BV encoding introduces no approximations (holds for integers; may fail for IEEE 754 NaN propagation).

When any condition is violated, SEMREC may report EQUIVALENT for a genuinely divergent pair (false negative). In practice, condition C3 is the most frequently violated.

In the checked artifact, this boundary is executable rather than aspirational: `src/ub_oracle/completeness.py` brute-force enumerates the finite integer fragments against the concrete C-UB predicates and requires the symbolic oracle to agree exactly, while `formal/CompletenessBoundary.lean` proves the published complete-vs.-may-abstain class partition is total and disjoint.

Full proofs of all theorems and their supporting lemmas appear in Appendix A.

3 Implementation

SEMREC is implemented in ~ 116 K lines of Python, with the core verification pipeline comprising ~ 23 K lines across the parser frontends, IR lowering, product builder, SMT encoder, and solver interface. The test suite includes targeted real-compiler regression tests for every confirmation mode.

Adoption surfaces. The artifact is packaged as both a research harness and a usable guardrail: `cross-lang-verify` ingests pair-aware manifests, emits text/JSON/SARIF, supports triage, suppression baselines, caching and local dashboards, and is wrapped by a GitHub Action, VS Code diagnostics shim, playground and plugin SDK. Even the launch surface is generated from the verifier rather than hand-written: `scripts/build_readme_demo.py` runs the real CLI on `examples/readme_demo_units.json` and renders the README animation only when the C $\rightarrow$ Rust signed-overflow witness is confirmed by `clang/UBSan` and `rustc`, while the safe control is discharged by the interval pre-pass. Thus the public demo is a checked artifact, not a marketing mock-up. The longer README-linked video is generated the same way: `scripts/build_demo_video.py` first replays the checked-in `c2rust nginx_rate` extraction unit on a CWE-369-class witness with `clang/UBSan` and `rustc`, then confirms the C $\rightarrow$ Go teaser case through the same harness; if either replay fails, the MP4 and poster are not rendered. A separate launch-readiness gate (`scripts/validate_launch_readiness.py`, exposed as `make launch-check`) re-runs the same compiler-backed demo, validates the SARIF finding, and checks that the outreach, contribution, issue-template, and talk/blog artifacts cite only reproducible repository commands.

Parser frontends. Each language has two parsing backends: `tree-sitter-c` 0.24 and `tree-sitter-rust` 0.24 [24] as primary, with hand-written Pratt-parsing recursive-descent parsers [25] as fallback (`src/c_parser.py`, `src/rust_parser.py`). Failures concentrate on macro-heavy code and complex generics.

Compiler-IR evidence and localization. The implementation also ingests the compilers' own IRs for debugging and artifact evidence. `ir_ingest.py` recovers C signatures, storage classes, and parameter names from `clang -Xclang -ast-dump=json`, and recovers Rust signatures and move/drop ownership facts from `rustc -emit-mir`. The new `ir_diff.py` localizer compares normalized facts from those real IR dumps: for the signed-overflow witness it points to the C `BinaryOperator` whose `qualType` is signed `int` and the Rust MIR statement that is either a defined `wrapping_add` or an `AddWithOverflow` plus

overflow assertion. `make ir-diff-check` replays this localization against live `clang` and `rustc`; if the toolchains are absent, no IR evidence is fabricated.

Shared IR. Both ASTs are lowered to a shared typed SSA IR [26] (`src/ir/`) with 36 instruction types organized into arithmetic (`BinaryOp` with 18 opcodes annotated with `OverflowBehavior`), unary (`UnaryOp`, 4 opcodes), comparison (16 predicates including ordered/unordered float), cast (12 kinds), memory (`Load`, `Store`, `Alloca`, `GEP`), control flow (`Branch`, `Switch`, `Return`, `Call`), and SSA (`Phi`, `Select`, `ExtractValue`). The type system includes `IntType` (with width and signedness), `FloatType` (F32/F64), `PointerType` (with provenance tags: `UNIQUE`, `SHARED`, `RAW`), `StructType`, `EnumType`, and `FunctionType`.

Loop handling. Loops are unrolled up to the configurable bound K (default 32). Phi nodes are threaded across iterations via a `prev_defs` mapping in the encoder. This is the fundamental limitation of the bounded approach: divergences requiring more than K iterations are missed. A K -sensitivity analysis (Section 4.6) is planned to empirically justify $K=32$.

SMT backend. SEMREC targets Z3 [17] via the Python bindings. The primary theory is `QF_BV` (quantifier-free bitvectors); for pointer-manipulating code, the solver extends to `QF_ABV` (arrays of bitvectors) with byte-addressable memory, and floating-point contraction uses Z3’s IEEE-754 binary64 FP theory with explicit round-to-nearest-even. The `to_smt_lib()` method on `ProductProgram` can emit SMT-LIB2 scripts for external solver use.

Memory model. For pointer-manipulating code, SEMREC incorporates Andersen-style flow-insensitive points-to analysis [27] (`src/smt/points_to_analysis.py`), Rust ownership non-aliasing axioms (for each pair of simultaneously live `&mut T` references p, q : $p \neq q$), and type-based alias analysis following C11’s strict aliasing rule (§6.5/7). Memory operations include alignment checks ($addr \& (align - 1) = 0$), bounds checks against known allocations, and use-after-free detection.

Mechanized product and class-specific evidence. The Lean development now covers the product-program witness boundary itself: raw product-run facts (payload, UB-reached flag, target return code, and consequence flag) are converted to the pack-parametric observation, and any emitted counterexample is proved to preserve that payload and observation while being a genuine UB-rooted divergence. This keeps the checked theorem at the same abstraction boundary as the executable verifier rather than merely testing the packaging code.

The strict-aliasing class is confirmed by optimization exploitation rather than a sanitizer: the same generated C type-pun source is compiled at `-O0` and `-O2 -fstrict-aliasing`; if both runs are clean but print different observables, the source behavior is under-determined. The Lean development mechanizes the class-specific pigeonhole step: two distinct C observables cannot both equal one deterministic, defined target observable, so at least one legal C build diverges from the target and the generic product-program soundness theorem applies. The same Lean file now also mechanizes the pointer-provenance TRAP confirmation shape: the generated C witness has valid data input but forms an out-of-provenance pointer, UBSan traps on that concrete run, and the target keeps a checked index and returns deterministically. In the recorded-observable abstraction, that trap versus defined target outcome is exactly the product-program consequence.

Step 131 makes the completeness boundary itself a checked object. The new `formal/CompletenessBoundary.lean` module imports the product-program theorem, classifies the integer classes (`signed_overflow`, `shift_oob`, `div_by_zero`, `intmin_div_neg1`) as complete on their declared finite ranges, and classifies memory-shape/FP rows such as `array_oob`, `strict_aliasing`, and `fp_contraction` as sound but may-abstain. Lean proves this partition is exhaustive and non-overlapping, and the Python driver builds it through Lake so imports and stale proof artifacts cannot hide a broken theorem. The result deliberately does not overclaim: finite-range no-false-negative evidence still comes from the independent brute-force/Z3 agreement check in `completeness.py`.

Step 136 makes the extension surface a checked object too. `formal/SPIContract.lean` states the target-pack and divergence-oracle SPI as Lean typeclasses: a conforming pack must prove that its `ProductSoundness` defined-return predicate agrees with its declared finite data, that canonicalization accepts only declared defined outcomes, and that product-level positive claims route through the pack-parametric `pack_oracle_sound` theorem; a conforming plugin must expose the real Python method sur-

face (`applies_to`, `find_divergence`, `confirm`) and prove that found/confirmed witnesses are product-program violations. The companion Python checker then ties the Lean source to the live registry and executable pack-conformance suite, so adding a target pack requires both a Lean instance and the concrete harness obligations.

Step 137 closes the corresponding engineering loophole for oracle classes. `ub_oracle.soundness_gate` enumerates the live plugin registry after all built-in modules have registered themselves and requires a static `SoundnessStatement` for each exact $(source, target, class)$ instance. The statement binds the oracle to its expected confirmation mode (for example `trap_vs_defined`, `optimizer_exploited`, `defined_divergence`, `model_level_divergence`, or `source_defined_target_ub`), the source-definedness premise it claims, and a traceability or Lean theorem reference. The gate then calls the oracle’s real `find_divergence` implementation on a concrete witness unit and checks that the emitted counterexample has the declared pair, class, source-definedness, non-empty source/target programs, and witness prose. CI runs `make soundness-check`; the focused negative-control test appends an otherwise-valid plugin with no statement and proves the gate rejects it. Thus a new oracle cannot enter the registry as an unreviewed soundness claim, even before the heavier real-compiler confirmation jobs run.

Step 129 turns this mechanization into an executable artifact, Step 132 makes it durable in the counterexample format, and Step 138 closes the canonicalization ambiguity in that certificate hash. The repository pins Lean 4.14, builds `formal/VerifiedChecker.lean` with `lake build verified-checker`, and exposes it through `cross-lang-verify -verified-check`. Every confirmed source-UB witness now serializes a *proof certificate*: canonical JSON binds the certificate to the source/target snippets, concrete inputs, observed replay transcript, and confirmed bit; a typed, length-prefixed verdict-layer preimage binds the recorded $P \wedge T \wedge C$ observation, theorem name, issuer, and counterexample hash; and cache hits rehydrate the same certificate rather than a bare verdict. The cache key is content addressed by the canonical unit hash, the real toolchain fingerprint, and the semantics version; the executable `prove_cache_equivalence` check cold-runs a fresh cache, warm-runs the same units, and requires the normalized verdict/counterexample signature hashes to match exactly. Step 146 adds the analogous operational guard for memory: `ReexecHarness` can run compiler and program subprocess trees under a supervised `-max-rss-mb` peak-RSS cap. The supervisor samples the whole process tree instead of using address-space rlimits (which break sanitizer shadow-memory reservation), and a cap hit is recorded as resource exhaustion, never as UBSan or target-language semantic evidence. The executable `prove_memory_bounded_equivalence` check runs the same real units unbounded and capped, then rejects the proof unless the verdict-layer signatures match and no subprocess exhausts the RSS budget. The same version binding now extends beyond cache keys: every environment-dependent result artifact (`confirmations.json`, `minimized.json`, `attack.json`, and `timings.json`) records a `toolchain-provenance/v1` payload with compiler/runtime paths for audit and exact first-line version fingerprints for `clang`, `rustc`, `go`, and registered target runners. Replay-check modes compare only those version fingerprints (not machine-local paths) and fail fast on missing provenance, unknown probes, or any version drift. `formal/HashStability.lean` proves this verdict canonicalization injective: two distinct verdict payloads cannot produce the same canonical preimage, so equality of final certificate hashes is reduced to SHA-256 rather than to an ambiguous field-boundary encoding. The offline certificate checker rejects schema drift, rebinding, corruption, or a non-violating observation without re-running compilers. The Lake-built checker then imports `ProductSoundness`, calls the proved `productViolated` predicate directly, and rejects a claimed source-UB DIVERGENT verdict unless the certificate-carried facts satisfy $P \wedge T \wedge C$. Its compile-time theorem guards include `oracle_sound`, `proof_carrying_counterexample_sound`, and `verdict_canonicalization_injective`; removing or renaming those theorem guards breaks the build. This does not replace compiler re-execution—the facts still come from `clang/rustc/runtime` runs, and the hash is an integrity binding rather than an adversarial signature—and it is deliberately not applied to safe→safe defined-divergence reports. It does, however, make the shipped witness itself sufficient to re-check the final UB-rooted positive-claim inference offline.

Step 139 verifies the minimizer at that same boundary. The executable minimizer still shrinks only by trying simpler input values and accepting a step only after the real ground-truth harness re-confirms the reduced witness; for UBSan-backed classes it also requires the same normalized diagnostic category, preventing an `INT_MIN/-1` witness from collapsing into a division-by-zero witness or a too-large shift into a negative shift. The new certificate records the accepted reduction chain, the category, and the final local-minimality check. Lean proves `minimizer_preserves_divergence`, `minimizer_reduction_steps_preserve_divergence`, and `minimizer_certificate_sound`: any accepted step chain whose final observation was harness-confirmed still denotes a UB-rooted divergence. This is a preservation theorem, not a global-optimality theorem; the latter would be the wrong claim for

a deliberately local delta-debugging ladder.

Finally, Step 140 turns the registry into a reviewable compendium. `ub_oracle.soundness_compendium` joins `SOUNDNESS_STATEMENTS`, `traceability.CLAIMS`, and the `ProductSoundness` theorem contract to generate `docs/SOUNDNESS_COMPENDIUM.md`: one row per oracle instance, with its confirmation mode, source-definedness premise, theorem/claim references, and the concrete witness unit that exercises the real `find_divergence` path. A freshness check rejects stale prose or dangling theorem references.

Step 130 adds an independent proof-assistant cross-check rather than another test. The Coq development `formal/CoreSoundness.v` deliberately imports neither the Lean file nor any generated artifact: it redefines the recorded-observable triple, the product assertion $R = \neg(P \wedge T \wedge C)$, the target-pack observation function, and the product-run-to-counterexample constructor, then proves the same central facts (`oracle_sound_coq`, `oracle_decides_coq`, `rust_oracle_sound_coq`, and `product_program_preserves_divergence_w`). The Python driver runs `coqc` when available and otherwise reports an explicit source-contract-only status, so the artifact never labels an unrun Coq proof as kernel-checked.

Step 94 makes the submission process itself auditable. The new `ub_oracle.pre_review` gate generates `docs/PRE_REVIEW.md`, a hash-stable external-review packet with four independent reviewer lanes (PL soundness, security/UB, systems artifact, and empirical methods), 14 evidence items, and eight open-artifact release checks. The validator is deliberately toolchain-independent: it reads only committed metadata and fast registries, then verifies that every cited evidence path exists, every command resolves to a real repository entry point (`make`, `python -m`, `bash`, or `pdflatex`), every reviewer question is tied to evidence, the MIT licence and public `CITATION.cff` are present, the claims audit passes, and all 60 registered oracle instances have soundness statements. The checked packet currently has hash `59e1b040ffaf`; a dangling paper path, stale Make target, missing review lane, or unbacked release claim fails `make pre-review-check` before the PDF can be treated as ready for external red-team review.

Step 199 freezes the ecosystem contract rather than leaving it as social policy. `ub_oracle.ecosystem` defines `PUBLIC_API_V1`, writes `integrations/api_surface_v1.json`, and `make ecosystem-check` now enforces three compatibility obligations: every promised v1 symbol remains exported and live, the committed API snapshot is byte-fresh without a side-effecting regeneration step, and the shipped `cargo cross-lang-verify` shim produces the same JSON as the in-process library CLI. A SemVer-breaking removal is therefore a test failure, not a release-note convention.

4 Evaluation

We evaluate SEMREC around twelve research questions. All experiments run on a laptop (Apple M-series, 16 GB RAM) using Python 3 and Z3.

- RQ1** Can SEMREC detect UB divergences that are provably invisible to IR analysis and differential testing?
- RQ2** End-to-end accuracy on a general benchmark.
- RQ3** Sensitivity to the BMC bound K .
- RQ4** Counterexample quality.
- RQ5** Verification performance.
- RQ6** Comparison to differential testing.
- RQ7** Historical-CVE weakness replays across $C \rightarrow \text{Rust}/\text{Go}$.
- RQ8** Does every divergence class transfer across *multiple* target languages and confirm against *real* compilers (`clang/rustc/go/clang++/ocamlpt`)?
- RQ9** At which optimization level does each no-sanitizer-traps UB class first change observable output?
- RQ10** Given one C source ported to ≥ 3 safe targets, can the oracle flag the minority target on live output?
- RQ11** Can SMT encodings of the integer-UB family and FP-contraction class be *proved* against their specifications and beat or complement enumeration at finding rare witnesses, while agreeing with independent enumeration semantics on a large sampled differential check?
- RQ12** Does the artifact exercise actual c2rust machine output, rather than hand-written c2rust-style translations, and can it connect that pipeline to public GitHub $C \rightarrow \text{Rust}$ port families?

4.1 UB-Invisible Benchmark (RQ1)

Benchmark construction. We construct a benchmark of 30 divergent C/Rust function pairs that are *provably invisible* to both IR-level analysis and differential testing. Each pair exposes a specific UB class where C has undefined behavior and Rust wraps, masks, or panics. Crucially, on x86 hardware the C

Table 2: UB-invisible benchmark: 30 divergent pairs + 10 equivalent controls. SEMREC achieves 100% recall; all baselines score 0%.

Method	Recall	Precision	F1
SEMREC	30/30 (100%)	93.8%	96.8%
Diff testing (naïve, 10K)	0/30 (0%)	—	—
Diff testing (UB-aware)	0/30 (0%)	—	—
IR comparison	0/30 (0%)	—	—

Table 3: Per-UB-class breakdown. SEMREC achieves perfect recall across all 6 classes.

UB class	Pairs	Detected	Recall
Overflow	9	9	100%
Shift	5	5	100%
Division	5	5	100%
Cast	5	5	100%
Compound	5	5	100%
Negate	1	1	100%
Total	30	30	100%

undefined behavior manifests as the same wrapping operation that Rust specifies—so differential testing (even with edge-case inputs) observes identical outputs. Additionally, LLVM compiles both versions to identical IR (both emit plain `add i32` etc.), so IR-level tools cannot distinguish them. We include 10 equivalent pairs as negative controls.

The 30 divergent pairs span 6 UB classes:

- **Overflow** (9): signed `add/sub/mul`, cascading chains, `INT_MAX + 1`
- **Shift** (5): left/right by ≥ 32 , variable shift, negative shift, shift-and-mask
- **Division** (5): `INT_MIN / (-1)`, `INT_MIN % (-1)`, negative truncation, division by zero, promotion truncation
- **Cast** (5): sign extension, unsigned-to-signed overflow, implicit narrowing
- **Compound** (5): add-then-shift, multiply-then-divide, mixed promotion chains
- **Negate** (1): `-INT_MIN`

Baselines. We implement three baselines:

1. **Differential testing (naïve, 10K inputs)**: uniform random inputs; compares C output (compiled -O0 x86) to Rust output.
2. **Differential testing (UB-aware, edge cases)**: targeted inputs including `INT_MIN`, `INT_MAX`, ± 1 , 0, shift amounts near bit-width.
3. **IR comparison**: compile both to LLVM IR at -O0 and compare instruction-by-instruction.

All three baselines achieve **0/30 recall**: they report all 30 divergent pairs as equivalent. This confirms that the divergences are genuinely invisible at the IR and execution levels. This negative result does not preclude using compiler IR as *evidence*: the new IR localizer intentionally compares source-level compiler products (`clang` AST and `rustc` MIR), not erased LLVM equality, and uses them to explain where the already-found semantic split occurs.

Results. Table 2 summarizes the results. SEMREC detects all 30 UB divergences with 100% recall. Precision is 93.8% (2 false positives on equivalent pairs where the oracle’s UB assumptions are not gated by conditional guards—a known limitation). F1 is 96.8%.

Table 3 shows that SEMREC achieves perfect recall across all 6 UB classes, including division UB (enabled by the SDIV/SREM encoding fix described in Section 2).

Why baselines fail. On x86, C signed overflow *wraps* identically to Rust: the hardware executes the same `ADD` instruction regardless of language. Differential testing therefore sees identical outputs for every input, including edge cases. IR comparison fails because LLVM erases the signed/unsigned distinction: the C compiler adds an `nsw` metadata flag but produces the same opcode. Only source-level analysis, which encodes the C11 UB semantics as SMT assumptions, can detect these divergences.

Table 4: Migration-scale corpus census, generated directly by `paper_scale_section.py` from `corpus_census`.

Quantity	Count
Programs / distinct program hashes	60,000 / 60,000
C LOC / target LOC	499,200 / 544,800
Total LOC	1,044,000
C→Rust / C→Go programs	32,000 / 28,000
Divergent / equivalent labels	29,600 / 30,400
Label-balance delta	800 (1.33%)

Table 5: Scale-corpus family mix; every row is counted over the full 60k-program population rather than sampled.

Class	Pattern	Family	Count
<code>div_by_zero</code>	division by zero	UB-rooted divergent	9,600
<code>oob_read</code>	out-of-bounds read	UB-rooted divergent	9,600
<code>oversized_shift</code>	oversized shift	UB-rooted divergent	6,400
<code>safe_add</code>	safe addition	defined equivalent	9,600
<code>safe_mod</code>	safe modulo	defined equivalent	9,600
<code>safe_mul</code>	safe multiplication	defined equivalent	4,800
<code>safe_shift</code>	safe shift	defined equivalent	6,400
<code>signed_overflow</code>	signed overflow	UB-rooted divergent	4,000

4.2 General Benchmark Suite

The general benchmark comprises 511 C/Rust pairs organized into three tiers:

- **Core** (52 pairs): hand-crafted with verified ground truth, covering canonical divergence patterns.
- **Combined** (212 pairs): core plus generated and extended cases across 11 categories.
- **Full** (511 pairs): all pairs, including real-world library functions from `musl-libc`, `zlib`, `libsodium`, `SQLite`, and the Linux kernel.

4.3 Migration-Scale Corpus (RQ1b)

Hand-picked witnesses do not establish migration-scale trust. The artifact therefore includes a deterministic generator (`src/ub_oracle/large_scale_study.py`) that enumerates **60,000** genuinely distinct $C \rightarrow \{Rust, Go\}$ programs totaling **1,044,000** counted source+target LOC. The generator bakes *defined* operands into each source file and reads only the UB-triggering operand from `argv`; the census is therefore a structural claim, not a sampling estimate. Every program hash is unique (60,000/60,000), and the declared labels are deliberately balanced: 29,600 divergent vs. 30,400 equivalent (1.33% delta).

The live part remains honest about cost: CI can execute a deterministic seeded sample rather than compiling the full million-LOC corpus on every run, and `available=false` is reported instead of invented evidence when a required toolchain is missing. The same verdict-layer hash discipline used by the parallel, sharded, distributed-manifest, memory-bounded, and result-store checks excludes timing and host metadata from semantic reproducibility.

On the current full toolchain, `make large-scale` confirms 7 divergent and 5 equivalent sampled items with 12/12 observed labels matching the declared ground truth (verdict-layer hash prefix `5ec6c935c7b0`).

4.4 Tier-1 c2rust-Output Corpus (RQ12)

The flagship population for $C \rightarrow Rust$ migration is not hand-written Rust but machine output from `c2rust`. We therefore added a Tier-1 corpus whose source side consists of 12 compact extraction units from distinct real library families (`musl`, `zlib`, `SQLite`, `libpng`, `nginx`, `curl`, `Lua`, `OpenSSL`, `git`, `Redis`, `bzip2`, and `XZ Utils`). Each C unit is translated by the actual `c2rust 0.22.1` binary, and the generated Rust files—with `#[no_mangle]`, `unsafe extern "C"`, and `::core::ffi` signatures—are checked in under `experiments/c2rust_corpus/generated`. The artifact records hashes of both source and generated Rust, validates the IR unit, runs the verifier with ground-truth confirmation disabled (so symbolic

Table 6: Tier-1 c2rust-output corpus. Every Rust artifact is generated by c2rust 0.22.1; make c2rust-corpus-check regenerates the Rust and byte-checks the verifier-backed verdict layer; the focused corpus test also compiles all 24 source/generated artifacts.

Divergence class	Units	Verdicts	Evidence
Signed overflow	6	3 candidate / 3 no-divergence	Z3 witness or interval pre-pass
Shift out of range	3	2 candidate / 1 no-divergence	Z3 witness or safe shift range
Division by zero	2	2 candidate	zero-divisor witness
INT_MIN/-1	1	1 candidate	unique signed-div overflow witness
Total	12	8 candidate / 4 no-divergence	12 generated Rust files compile

Table 7: Verification accuracy by category on the full 511-pair benchmark. Accuracy ranges from 0% (Other) to 100% (Shift), reflecting honest pipeline coverage limitations.

Category	Pairs	Correct	Accuracy	Dominant failure mode
Shift	17	17	100.0%	—
Arithmetic	49	46	93.9%	—
Division	21	19	90.5%	—
Cast	28	22	78.6%	Float-to-int edge cases
Real patterns	26	16	61.5%	Complex expressions
Float	15	9	60.0%	NaN, rounding
Error handling	21	4	19.0%	Result/Option mapping
Compound	27	11	40.7%	Multi-op chains
Bitwise	46	18	39.1%	Complex bit patterns
Control flow	21	5	23.8%	Nested branching
Loop	8	2	25.0%	Accumulator overflow
C2Rust	38	7	18.4%	Struct/trait lowering
Enum	20	3	15.0%	Discriminant mapping
Memory/pointer	18	2	11.1%	Pointer semantics gap
Other	156	0	0.0%	IR lowering failure
Full (511)	511	181	35.4%	
Combined (212)	212	58	27.4%	

witnesses are reported as CANDIDATE, never overclaimed as confirmed divergence), checks that a fresh c2rust run byte-for-byte regenerates the checked-in Rust, and compiles every C/Rust artifact with the local clang/rustc toolchain.

This corpus replaces the weak “c2rust-style” claim with a reproducible machine translation check. It is still intentionally scoped: these are single-function extraction units rather than whole build systems, so whole-project c2rust ingestion remains an engineering frontier. The important credibility property is that every reported row is tied to exact generated Rust bytes and a verifier record in experiments/c2rust_corpus/results.json.

GitHub-mined C→Rust port sample. To keep the real-world funnel live beyond the fixed c2rust corpus, the artifact also includes a deterministic GitHub-port miner (src/ub_oracle/github_port_miner.py). The miner parses GitHub repository-search JSON, scores repositories by Rust evidence, C-lineage evidence, port/rewrite language, and popularity, and seeds the current run with three public port/rewrite families: uutils/coreutils, triffectatechfoundation/sudo-rs, and triffectatechfoundation/zlib-rs. For reproducibility and licensing, the checked-in source files are compact extraction units derived from those project families rather than copied upstream files. The resulting sample has two symbolic witnesses (signed overflow and division by zero) plus one interval-proved safe control; make github-port-mining-check byte-checks the candidate ranking, source hashes, verifier verdicts, and pre-pass result in experiments/github_ports/res. A separate skip-gated test compiles every sample as real C/Rust code and confirms a divergent sample against live clang/UBSan+rustc when those toolchains are present.

Table 8: BMC bound sensitivity on 38 benchmark pairs.

K	Accuracy	Δ	Observation
8	55.3%	—	Baseline
16	60.5%	+5.3pp	Additional loop-depth divergences caught
32	63.2%	+2.6pp	Saturation point
64	63.2%	+0.0pp	No change
128	63.2%	+0.0pp	Confirms saturation

4.5 General Accuracy (RQ2)

Table 7 reports accuracy by category. End-to-end accuracy on the full benchmark is 35.4% (181/511). On the 212-pair combined tier, accuracy is 27.4% (58/212), with a 95% Clopper–Pearson confidence interval of [21.8%, 33.7%]. The strongest categories are shift (100%), arithmetic (93.9%), and division (90.5%); the weakest involve struct, iterator, and string handling where IR lowering fails entirely. The improvement over a naïve bitvector approach stems from tree-sitter integration, improved IR lowering, and the σ -bridge coercion framework.

Honest assessment. These numbers are low compared to mature single-language verifiers, and we do not attempt to disguise this. The primary bottleneck is *pipeline coverage*, not verification soundness: IR lowering fails on struct manipulation, dynamic dispatch, and macro-generated code. When restricted to pairs where the pipeline completes without error, accuracy rises substantially (e.g., 83.3% on arithmetic in the combined tier, 100% on shifts). The tool is honest about its limitations: pairs that fail parsing or lowering are reported as UNKNOWN, not silently classified.

An ablation study on 10 representative pairs confirms the σ -bridge framework’s contribution: full pipeline achieves 80% accuracy vs. 40% without σ -bridge coercions (plain bitvector semantics), a $2\times$ improvement from coercion insertion alone.

4.6 K -Sensitivity (RQ3)

Table 8 shows that accuracy increases monotonically with K and saturates at $K=32$ (63.2%). Beyond $K=32$, additional unrolling yields no further accuracy gains on this benchmark, confirming the saturation hypothesis.

4.7 Counterexample Quality (RQ4)

Of the pairs where SEMREC returns a SAT verdict (divergent), counterexamples include concrete input values that trigger the divergence. For example, on an `add` pair, the counterexample $a = 2,147,483,647, b = 1$ immediately identifies signed overflow; on a division pair, $a = INT_MIN, b = -1$ identifies the `INT_MIN/-1` case.

4.8 Performance (RQ5)

Median verification time is 3.8ms per pair, with mean 308.1ms (skewed by a few complex pairs) and P95 at 53.4ms. Formula complexity—not source size—dominates verification time. The dedicated scalability grid is more revealing because it times the real oracle searches behind every registered class/pair cell: on the checked-in timing artifact the class/pair rows have P50 1.39ms and P95 9.26ms, with a single intentional outlier, the IEEE-754 FP-contraction row, at 9.84s. Step 145 turns this from a paper-only observation into an executable regression gate: `make perf-check` regenerates the deterministic grid and then measures each class/pair row, each supported integer-width row, and the SMT backbone probes under per-row nearest-rank P50/P95 latency budgets. The ordinary bit-vector/dataflow rows are budgeted at 0.25s/0.50s, the known FP-theory slow path at 15s/20s, width-scaling rows at 0.25s/0.50s, and SMT overflow probes at 0.05s/0.10s. Thus a solver blow-up is a CI failure, not an anecdote discovered after the fact.

Table 9: SemRec vs. differential testing on 10 controlled pairs.

Method	Accuracy	Unique finds	Median time
Differential testing (10K inputs)	100%	2	18.6 ms
SEMREC (bounded, $K=32$)	70%	1	143.5 ms

Table 10: Historical-CVE weakness replay corpus. The artifact indexes 60 public NVD CVE ids, generates 120 per-CVE $C \rightarrow \{\text{Rust}, \text{Go}\}$ bundles, lint-checks every bundle, and live-confirms every distinct replay template against real compilers.

CWE family	Replay template	CVE ids	Targets
CWE-190	signed overflow	8	Rust, Go
CWE-191	signed underflow	8	Rust, Go
CWE-369	divide by zero	8	Rust, Go
CWE-125	out-of-bounds read	8	Rust, Go
CWE-787	out-of-bounds write	8	Rust, Go
CWE-476	null dereference	8	Rust, Go
CWE-758	oversized shift / C UB reliance	6	Rust, Go
CWE-682	INT_MIN/-1 boundary	6	Rust, Go
Aggregate		60	120 bundles

4.9 Comparison to Differential Testing (RQ6)

On a controlled subset of 10 pairs, we compare SEMREC to property-based differential testing (10K random inputs per pair):

Differential testing achieves higher accuracy on this small subset, but its advantage is sample-dependent and it provides no equivalence guarantee. SEMREC’s unique advantage is formal: SEMREC found a `saturating_add` divergence that 10K random inputs missed, while differential testing found `rotate_right` and `safe_divide` divergences that SEMREC missed due to IR lowering failures. In practice, the tools are complementary.

Limitations summary.

- **Bounded model checking:** equivalence holds only within loop bound K .
- **Parser coverage:** macro-heavy code and complex generics fail.
- **IR lowering:** struct manipulation, dynamic dispatch, and trait objects cause pipeline failures.
- **Intraprocedural only:** no function summaries or call-chain verification.
- **Memory model:** value-level oracles cover strict-aliasing (with a Lean-checked optimizer-exploitation lemma), `restrict`, and pointer-provenance UB (with a Lean-checked TRAP confirmation lemma); full lifetime/heap-shape divergences (use-after-free, dangling) are reported via dedicated memory-shape detectors rather than a single value witness.
- **Floating point:** contraction and `-ffast-math` reassociation are witnessed against real `clang`; signaling-NaN propagation remains approximated.

4.10 Historical-CVE Weakness Replays (RQ7)

To evaluate SEMREC on vulnerability patterns with public provenance, we extend the original CWE-tagged corpus into a **60-entry historical-CVE replay set**. Each row is an NVD-indexed CVE id whose primary weakness is one of eight CWE families that matter for C migration: signed overflow, signed underflow, divide-by-zero, out-of-bounds read, out-of-bounds write, null dereference, reliance on undefined behaviour, and incorrect-calculation boundary cases. We do not vendor third-party vulnerable source. Instead, each CVE id is tied to a from-scratch minimized witness for its CWE family and to generated $C \rightarrow \text{Rust}$ and $C \rightarrow \text{Go}$ reproduction bundles. A bundle succeeds only when the C/UBSan build traps on the witnessing input, the target run has a language-defined outcome, and a safe negative input runs cleanly.

Table 10 reports the deployed corpus. The important check is not string matching on CVE ids: for each family, SEMREC’s ordinary real-compiler re-execution path confirms the unique witness template, and the bundle generator can materialize the per-CVE C/Rust or C/Go script that a reviewer can run

outside the Python harness. This gives the paper a public-vulnerability index without claiming that the artifact redistributes or recreates proprietary vendor bug code.

Limitation: replay scope. The historical table is a positive, weakness-class replay benchmark. It is not a claim that the repository contains original vulnerable vendor sources for those CVEs. Specificity is instead measured by the balanced labelled corpus and by the safe inputs embedded in every generated reproduction bundle.

Advantage over baseline. The 70pp advantage (100% vs. 30% on UB-dependent divergences) highlights a fundamental limitation of pattern-matching and testing approaches: when UB is “benign” on the target platform, sampling-based and syntactic approaches miss divergences that require modeling the language specification. SEMREC’s semantic analysis operates on the *language specification* rather than compiled behavior, enabling it to flag divergences that are invisible to pattern-based or testing-based approaches on the deployment platform.

New capabilities exercised. Three of the 10 pairs required pipeline features beyond the original benchmark:

- **Struct layout analysis** (`union_type_punning`, `aliasing_violation`): SEMREC models C’s implementation-defined struct padding and union reinterpretation against Rust’s guaranteed layouts.
- **Integer promotion tracking** (`integer_overflow`, `bitshift_overflow`, `floating_point`): the pipeline now tracks C’s implicit integer promotion rules (e.g., `char` $\rightarrow$ `int` widening before arithmetic) that have no Rust analogue.
- **Preprocessor handling** (`string_handling`, `varargs_handling`): macro-expanded C source is normalized before IR lowering, resolving a prior limitation on macro-heavy code.

4.11 Cross-Pair Generality and Real-Compiler Confirmation (RQ8)

A divergence oracle is only as trustworthy as its weakest unconfirmed verdict. SEMREC therefore enforces a two-layer contract for *every* class and *every* language pair, materialized as the deterministic cross-pair regression matrix (`src/ub_oracle/regression_matrix.py`):

1. **Symbolic layer** (toolchain-free, byte-reproducible). One canonical unit per class is driven through each registered oracle; the artifact is regenerated in a fresh interpreter and asserted byte-identical in CI (`make matrix-check`).
2. **Ground-truth layer.** Each cell’s solver/dataflow/model witness is compiled and run against real `clang` (under `-fsanitize=undefined`, MemorySanitizer/auto-init padding checks, or a checked C contract/model-level build where sanitizer evidence is the wrong proof object), real `rustc`, real `go`, real `clang++`, real `ocaml`, and real `zig`; a cell is *confirmed* only when the appropriate mode’s evidence holds: sanitizer/contract trap, optimization disagreement, padding-byte initialization delta, defined-vs-defined output difference, static compiler diagnostic, or bounded allowed-execution-set gap.

Table 11 reports the matrix. Its C-rooted core spans **7 target languages/runtimes** and **58 cells** (C $\rightarrow$ Rust: 20 classes; C $\rightarrow$ Go: 18; C $\rightarrow$ Swift: 6, the argv-driven integer/memory core; C $\rightarrow$ OCaml: 4, a GC’d exception-based target; C $\rightarrow$ C++: 1, the C/C++-boundary shift class; C $\rightarrow$ WebAssembly: 4, wasmtime-executed value-or-trap integer semantics; C $\rightarrow$ Zig: 5, Zig `ReleaseSafe` wrapping/trap/bounds semantics). The Go $\rightarrow$ Rust and Rust $\rightarrow$ C pairs add the two non-forward-C cells described below and raise the total to **60 cells over 9 pairs**. On a host with the corresponding toolchains/runtimes, every attempted cell must confirm against real compilers, checked C contract builds, or a real runtime trap/value outcome—no verdict is taken on faith. Crucially, the matrix file contains *no per-language branches*: it discovers pairs and oracles from the plugin registry, so adding a target language or a divergence class makes the corresponding cells appear automatically, and the soundness obligation is discharged uniformly.

A reverse pair: defined Rust panic vs. target-side C UB (Rust $\rightarrow$ C). The reverse pair exercises the FFI direction: an idiomatic Rust implementation may depend on a *defined panic*, while a C lowering reintroduces undefined behaviour. The oracle Z3-finds the same unique signed division/remainder overflow input, `INT_MIN` op `-1`. In Rust, both `/` and `%` panic deterministically in every build profile; the

source therefore has a language-defined observable abort. The generated C target performs the operation at the exact `int32_t/int64_t` width, where C17 leaves the non-representable quotient undefined and UBSan traps. A new confirmation mode, `source_defined_target_ub`, compiles the Rust source through real `rustc`, runs it twice to prove deterministic definedness, then compiles the C target with `clang -fsanitize=undefined -fno-sanitize-recover=all` and requires a real sanitizer trap on the same input. This cell proves the framework is not merely “C source bad, safe target good”: it can also detect a translation that *loses* a source-language safety guarantee.

A safe→safe pair: defined-but-different (Go→Rust). Most C-rooted cells exploit an asymmetry in which *one* side (C) is undefined. The framework, however, is not tied to undefined behaviour at all—its real content is *observable divergence between two language semantics*. To demonstrate this we add the first pair in which *neither* side has any UB: Go→Rust. The oracle’s Z3 search rediscovers the `INT_MIN/-1` signed overflow as the witness, but now *both* languages *define* it—and they define it *differently*. Go specifies modular two’s-complement arithmetic, so `INT_MIN/-1` evaluates to `-2147483648` and the program exits 0; Rust specifies that signed division overflow is a guaranteed *panic*, so the faithful port aborts with exit 101. Both outcomes are language-defined; the *observable* behaviour (a value vs. a deterministic abort) differs—exactly the hazard a Go→Rust transpiler would silently introduce. A new confirmation mode, `defined_divergence`, captures this: it builds both ports, runs each *twice* for determinism, and confirms iff *both* return codes lie in their language’s defined set *and* the observable results differ (the remainder form `INT_MIN% -1` is handled distinctly—0 in Go, panic in Rust). The cell is gated on *both* toolchains (go and `rustc`) rather than the C path, and confirms live on each. This single addition shows the oracle, the product-program construction, and the matrix contract all generalize verbatim from “C-UB vs. defined target” to “two defined languages that simply disagree.”

Implementation-defined ports: bit-fields and out-of-range enums. The same `defined_divergence` discipline exposes a second, subtler hazard class in which the C program is *not* undefined but merely *implementation-defined*, so a “faithful” hand port silently picks a different yet equally legal behaviour. (i) **Bit-field packing.** A C `struct { unsigned a:3, b:5, c:8; }` is laid out by the ABI into a single packed 4-byte storage unit; the obvious port that gives each field its own `u8/uint8` occupies 3 bytes with a different in-memory image. The oracle Z3-selects nonzero field values, then compares the raw object images: C reports `size=4, bytes=ffff0000` against the port’s `size=3, bytes=071fff`—a divergence that surfaces the instant the struct is `memcpy`’d, hashed, or sent over the wire. (ii) **Out-of-range enum.** C permits casting any integer in the enum’s underlying-type range to the enum, retaining the value verbatim ((enum Color)3 prints 3); the idiomatic Rust `match`/Go `switch` port collapses unmatched values to a default variant (printing 0). The oracle minimizes the least integer past the last enumerator and confirms 3 vs. 0 live on `clang+rustc` and `clang+go`. Both classes are implementation-defined rather than UB-rooted, extending the catalogue’s reach to the “legal-but-divergent” band that ordinary sanitizers and UB checkers cannot see.

Indeterminate object bytes: uninitialized struct padding. The newest C-rooted row covers a class that is neither a scalar UB trap nor a normal value mismatch: whole-struct byte serialization after assigning fields. For a compiler-asserted layout such as `struct P { uint8_t tag; uint32_t value; }`, the three bytes between `tag` and `value` are padding. The C witness assigns both fields, copies all `sizeof(struct P)` bytes into a digest, and lets a runtime switch zero padding for the safe control. On MSan-capable hosts the source side is confirmed by a real `use-of-uninitialized-value` report; on this artifact’s arm64 Darwin host the same source is confirmed by real `clang -ftrivial-auto-var-init=pattern` vs. `=zero`: the digest changes, while the explicit-zeroing control matches the zero build. The Rust and Go ports serialize fields into zero-initialized byte arrays at the compiler-validated C offsets, so their padding is deterministic. The divergence zoo now includes two live-confirmed exhibits for this class (C→Rust and C→Go), and the content-hashed zoo index is regenerated as `29a67003bcb5a9c3`.

Generalization v2 and per-pair soundness. The matrix counts registered cells; the V2 generalization study asks whether the new pair mechanisms transfer under live execution. It selects one representative case for each new direction—C→Zig, C→C++20, Rust→C, C→WebAssembly, Go→Rust, and C→OCaml—and runs the pair’s own registered oracle and confirmation discipline rather than a bespoke benchmark path. On hosts with the corresponding tools, the positive witness must confirm against real compilers/runtimes and the safe control must not confirm; on hosts without Zig or `wasmtime`, those rows are reported as unavailable rather than simulated. The paired soundness checker then verifies that

Table 11: Cross-pair regression matrix coverage. “D” = a symbolic divergence witness is found and (with toolchain/runtime) confirmed against real compilers; “-” = class not implemented for that pair. The C-rooted matrix now contains 58 cells, including five C→Zig **ReleaseSafe** cells and four C→WebAssembly value-or-trap cells; one safe→safe Go→Rust cell (defined-but-different INT_MIN/-1: a value in Go, a panic in Rust) and one Rust→C reverse cell (defined Rust panic vs. C UBSan-trapped UB) raise the total to **60 cells over 9 pairs**. The C→C++ column carries the single C/C++-boundary class whose witness (1 < 31) is UB in C but defined in C++20; the C→OCaml column carries the integer/memory core, witnessed against real `ocaml` (division-by-zero and out-of-bounds raise `Division_by_zero` / `Invalid_argument`; `Int32` arithmetic is exactly modular), and the C→Zig column is witnessed against real `zig build-exe -O ReleaseSafe`; the C→WebAssembly column is WAT executed by real `wasmtime`, where traps are accepted only when `stderr` confirms a wasm trap.

Divergence class	C→Rs	C→Go	C→Sw	C→MI	C→C++	C→Wasm	C→Zig	Go→Rs	Rs→C
Signed overflow	D	D	D	D	-	D	D	-	-
Shift ≥ width	D	D	D	-	-	D	D	-	-
Division by zero	D	D	D	D	-	D	D	-	-
INT_MIN / -1	D	D	D	D	-	D	D	D	D
Array out-of-bounds	D	D	D	D	-	-	D	-	-
Unsequenced read/write	D	-	-	-	-	-	-	-	-
Uninitialized read	D	D	D	-	-	-	-	-	-
Uninitialized padding	D	D	-	-	-	-	-	-	-
Strict aliasing	D	D	-	-	-	-	-	-	-
VLA bound	D	D	-	-	-	-	-	-	-
Float→int overflow	D	D	-	-	-	-	-	-	-
-ffast-math reassoc.	D	D	-	-	-	-	-	-	-
restrict violation	D	D	-	-	-	-	-	-	-
Pointer provenance	D	D	-	-	-	-	-	-	-
Overlapping memcpy	D	D	-	-	-	-	-	-	-
longjmp to exited VLA	D	D	-	-	-	-	-	-	-
Bit-field layout	D	D	-	-	-	-	-	-	-
Out-of-range enum	D	D	-	-	-	-	-	-	-
FP contraction	D	-	-	-	-	-	-	-	-
Relaxed atomics ordering	D	D	-	-	-	-	-	-	-
1 < 31 (C/C++)	-	-	-	-	D	-	-	-	-
Matrix cells	20	18	6	4	1	4	5	1	1

each new pair has a registry entry, expected confirmation mode (TRAP, SOURCE-DEFINED-TARGET-UB, or DEFINED-DIVERGENCE), target-pack resolution where applicable, a symbolic positive witness, and a symbolic negative control. Thus the paper’s pair-level claims are executable artifacts, not prose beside the implementation.

Labelled precision/recall. Independently of the matrix, a labelled divergence study (`experiments/ub_divergence`) scores the oracle on 29 cases spanning 19 anchor classes (23 positive, 6 negative-control families). The oracle attains **precision and recall of 1.0** (tp = 23, fp = 0, fn = 0, tn = 528 over the expanded negative population). On the same UB-invisible classes a differential fuzzer run for **200,000 trials per case** finds *nothing*: the gap is total, exactly because the C undefined behavior is “benign” on the deployment platform and only the source-semantics oracle can see it. Every number in this subsection is regenerated from content-hashed real-compiler runs by the checked-in experiment scripts.

4.12 Optimization-Level Onset of Latent UB (RQ9)

The three OPT-mode classes that *no sanitizer can trap*—strict aliasing, `restrict` violation, and `-ffast-math` reassociation—share a subtle property: at `-O0` the program behaves exactly as a naive reader expects, and the hazard only materializes once the optimizer begins to *rely* on the undefined or under-specified behaviour. The optimization-level sweep (`src/ub_oracle/opt_sweep.py`) makes this precise. For each such class it takes the oracle’s own Z3-found witness, compiles the witnessing C source at `-O0`, `-O1`, `-O2`, `-O3` (keeping any class-specific licence flag, e.g. `-ffast-math`, fixed at every level), runs each build on the witness input, and reports the *onset level*: the first level whose observable output

Table 12: Optimization-level sweep on real `clang17`. Each cell is the program’s observed stdout at that -O level; “onset” is the first level that diverges from the -O0 baseline. All three no-sanitizer-traps classes are latent at -O0 and surface at -O1.

Class	-O0	-O1	-O2	-O3	Onset
Strict aliasing	0	1	1	1	-O1
<code>restrict</code> violation	4	3	3	3	-O1
<code>-ffast-math</code> reassoc.	0	1	1	1	-O1

diverges from the -O0 baseline. Table 12 reports the result on real `clang17`: for all three classes the UB is latent at -O0 and surfaces at -O1, flipping the program’s printed value. This is a reviewer-legible, reproducible demonstration that the divergence is a genuine consequence of standard, conforming optimization—not an artefact of any single build.

4.13 N-Language Consistency: Majority Voting Across Live Ports (RQ10)

Pairwise oracles answer “does target T diverge from C?”. A migration shop, however, often maintains *several* candidate ports at once and wants a sharper question: given one C source compiled to ≥ 3 safe targets, *is any single target the odd one out?* The N -language consistency oracle (`src/ub_oracle/consistency.py`) answers exactly this. It builds the C program and each available target (Rust, Go, Swift, OCaml, Zig), runs them all on the oracle’s witness input, canonicalizes each observable outcome (a printed value, or an ABORT bucket for any trap/panic/uncaught exception), and applies majority voting: a target is *flagged* when it disagrees with the modal outcome. The result is purely live—every label is a real process exit, not a model prediction.

Two registered units make the behaviour concrete. On *out-of-range shift*, Rust’s `wrapping_shl_masks` the shift amount modulo the width ($1 \ll 32 \equiv 1 \ll 0 = 1$) whereas Go and Swift yield 0; the oracle flags **Rust** as the lone outlier among three safe languages—a genuine, defined-but-different divergence that no C-vs-one-target check would surface as “majority disagreement”. On *division by zero*, all three targets abort, so the verdict is *consistent*: a true negative that guards against over-flagging. Because the comparison reuses the same build harness and content-hashed outcomes as the pairwise path, its verdicts inherit the matrix’s reproducibility contract.

4.14 Target-Pack Conformance: a Mechanized Back-End SPI

Generality in SEMREC is *configuration, not code*: a new target language or runtime is a *target pack* (compiler/stager candidates, optional runner candidates, source/artifact suffixes, a total defined-outcome predicate, argv builders, and a class-resolution map) plus a table of per-class source emitters—no new harness or oracle logic. To keep that promise sound as packs proliferate, the conformance suite (`src/ub_oracle/pack_conformance.py`) discharges SPI obligations against *every* registered pack (Rust, Go, Swift, C++, OCaml, Zig, Wasm): the defined-outcome predicate is total and includes success while rejecting host errors; compiler/stager and runner candidates are well-typed; suffixes are well-formed; compile and run argv builders are deterministic and mention the right artifacts; and the class-resolution map only names known divergence classes. A pack that violates any obligation fails CI before it can contribute a single unsound cell to the matrix—turning “add a language or runtime” into a checklist the machine, not the reviewer, enforces.

The formal side now mirrors this executable gate. `SPISContract.lean` declares `TargetPackSPI` and `OraclePluginSPI` typeclasses and provides discharged instances for the seven registered target packs (Rust/Go/Swift/C++/OCaml/Zig/Wasm). The bridge checker (`src/ub_oracle/spi_contract.py`) verifies that every registered Python pack has a Lean instance, every executable obligation key appears in the Lean contract, and the formal plugin method list matches the actual `DivergenceOracle` interface; when Lake is available, CI builds `lake build SPISContract` and rejects stale or vacuous formal claims.

Table 13: Witness search, SMT vs. ordered enumeration, at 32-bit width (probe budget 2^{20}). For rare-witness classes enumeration spends its full budget without success while Z3 answers in sub-millisecond time; both paths are trivial on dense classes. Times are representative single-run wall-clock.

Class	SMT finds?	Enum finds?	Enum probes	Speed-up
Division by zero	yes	yes	1	—
Shift $\geq$ width	yes	yes	33	—
Signed overflow	yes	no	2^{20}	$\sim 100\times$
INT_MIN / -1	yes	no	2^{20}	$\sim 800\times$

4.15 SMT-Backed Decision Procedures: Proved Encodings and Bit-Precise Witnesses (RQ11)

For the integer-UB family—signed overflow, out-of-range shift, division by zero, and the unique INT_MIN/-1 overflow—we promote the *encoding itself* to a machine-checked artifact (`src/ub_oracle/smt_integer.py`). Each class carries two independent Z3 bit-vector formulas: an *operational* encoding (the formula the shipped oracle solves, e.g. `Not(BVSDivNoOverflow(a, b))` excluding $b = 0$) and a human-readable *specification* (e.g. $a = \text{INT_MIN} \wedge b = -1$). The tool then asks Z3 to prove the bi-implication valid over *every* input at the real bit widths: the negation *operational* $\not\leftrightarrow$ *spec* is **unsatisfiable** for all four classes at both 32 and 64 bits (and, as a width-parametric sanity check, at 4/6/8 bits where the entire input space is also brute-forceable). This is the equisatisfiability obligation discharged not on a sample but universally—the SMT path and the specification accept exactly the same witnesses.

The encoding is not merely elegant; it is *necessary* for the rare classes. Table 13 contrasts the SMT search with an ordered enumeration that scans the input space for a witness. For dense classes (a zero divisor, an out-of-range shift) both paths answer instantly. But for the classes whose witness is vanishingly rare—signed overflow under an ordered scan, and INT_MIN/-1 (a single input pair in 2^{64})—enumeration exhausts a multi-million-probe budget and finds *nothing*, while Z3 returns the witness in under a millisecond: a two-to-three-orders-of-magnitude speed-up that is the quantitative case for the symbolic encoding. Two independent agreement checks guard the decision procedure: an exhaustive differential cross-check confirms the two paths agree on *existence* at every fully enumerable width, and the Step 135 harness generates 10,000 deterministic concrete assignments across every shipped class and 32/64-bit width, then asks Z3 whether the operational formula can disagree with the independent enumeration predicate on any sampled assignment. Each class $\times$ width disagreement disjunction is UNSAT, and the SMT witness for INT_MIN/-1 coincides exactly with the pair the shipped operational oracle emits.

The same section of the artifact now includes a bit-precise floating-point certificate for the FP-contraction class (`src/ub_oracle/smt_float.py`). The solver asks Z3’s IEEE-754 binary64 theory for operands where $\text{fma}(a, b, c) \neq \text{round}(\text{round}(a \cdot b) + c)$ under round-to-nearest-even, while preserving the operational oracle’s finite-normal, bounded-magnitude, and cancellation constraints. The returned witness is not a decimal-only model: the artifact records operand bits `bffe1e223fe2d052, 3ff79adacef6bb27, 4006376792d200d8` and result bits `3d01fe159b5bd2ec` (single-rounding) versus `3d02000000000000` (double-rounding). A certificate replay reconstructs those IEEE bits as Z3 FP constants, recomputes the fused and unfused terms, and proves the negation of the bit-level contraction fact unsatisfiable. The shipped oracle then feeds the same operands to the existing real-compiler harness, which compiles one C source with `-ffp-contract=off` and `-ffp-contract=fast` and compares it with the deterministic Rust translation. Thus the FP row is simultaneously bit-precise at the solver boundary and grounded in live compiler output.

5 Related Work

Translation validation. Alive2 [14] validates LLVM IR $\rightarrow$ IR transformations but operates *after IR erasure*: the C11/Rust semantic distinction is already lost, so UB divergences are invisible. CompCert [16] provides end-to-end compilation correctness for C but targets a single language. Pnueli et al. [15] formalized translation validation as a proof obligation; SEMREC adapts this to the cross-language setting where the “translation” is a C $\rightarrow$ Rust migration.

IR-level verification. KLEE [11] performs symbolic execution on LLVM IR. SMACK [12] translates LLVM IR to Boogie. SeaHorn [13] uses constrained Horn clauses. CBMC [18] performs bounded model

checking on C. All operate on a single language or post-erasure IR. As demonstrated in Section 4.1, IR-level analysis achieves 0/30 recall on UB-invisible divergences: the signed/unsigned distinction that distinguishes C UB from Rust wrapping is erased by compilation. SEMREC is the only tool in this design space that detects these divergences.

Cross-language verification. The problem of verifying equivalence across language boundaries has received limited attention. Most existing work targets single-language settings or FFI ABI correctness. SEMREC is, to our knowledge, the first tool to perform source-level bounded equivalence verification for C→Rust migration with formal UB-aware soundness guarantees.

C→Rust migration tools. C2Rust [7] transpiles C to syntactically valid but heavily `unsafe` Rust. Crown [8] and Laertes [9] refactor C2Rust output toward safe idioms. LLM-based translators produce more idiomatic Rust but lack formal correctness guarantees. None perform source-level equivalence verification or detect UB divergences. Our Tier-1 corpus is therefore complementary: it does not replace c2rust, but validates selected c2rust-generated artifacts against the source-language UB oracle.

Formal verification for Rust. Kani [19] performs BMC on Rust (single-language). Prusti [20] verifies Rust against specifications. RustBelt [21] provides semantic foundations for Rust’s type system. RefinedC [22] verifies C with refinement types. These verify within a single language; SEMREC verifies *cross-language* equivalence.

Product programs. Barthe et al. [23] introduced product programs for relational verification of two runs of the *same* program. SEMREC extends this to two *different* programs in *different* languages with heterogeneous UB semantics, adding σ -bridge coercion nodes at semantic boundaries—the key mechanism enabling detection of IR-invisible divergences.

6 Conclusion

We presented SEMREC, a bounded source-level equivalence verifier for C→Rust migration that detects UB divergences *invisible to every existing verification approach*. On 30 IR-invisible UB divergences spanning 6 classes, SEMREC achieves **100% recall** while differential testing, IR comparison, and IR-level verification tools (KLEE, SMACK, Alive2) all score **0%**.

The key enabler is the σ -bridge product program construction, which keeps source and target semantic preconditions separate instead of normalizing both languages through a shared compiler IR. This preserves the signed/unsigned, division, shift-UB, contract, and scoped memory-ordering distinctions that compilation erases or under-specifies. The soundness theorem (Theorem 3) guarantees that UNSAT certifies equivalence on all well-defined inputs within the loop bound. When a positive witness is found, the compiler-IR localizer now complements the proof by pointing maintainers at the exact `clang`-AST operation and `rustc`-MIR statement whose semantics differ.

General-purpose accuracy (35.4% on 511 pairs) reflects parser and IR coverage limitations, not verification unsoundness. Where the pipeline completes, the formal guarantees hold. We view SEMREC as complementary to differential testing: testing covers broad program behavior but misses UB divergences by design; SEMREC catches exactly the bugs that testing cannot.

The tool is open-source, passes the targeted real-compiler regression suite, and is under active development. Future work includes interprocedural verification, struct/enum lowering, and integration with C2Rust for automated post-migration verification.

A Proof Obligations Referenced by the Soundness Sketch

This appendix records the four obligations used in the main proof sketch. The repository maps these obligations to the executable product-program checks and the mechanized soundness files cited in Section 2.3; the statements here are intentionally compact so the paper body can focus on the tool and evaluation.

Lemma 6 (Bitvector encoding faithfulness). *For every expression in the handled integer fragment, the QF_BV encoding evaluates to the same width-truncated value as the operational source or target semantics whenever the source-side well-definedness predicate holds.*

Lemma 7 (Well-definedness completeness). *For the handled C undefined-behavior classes, the SMT well-definedness predicate is true exactly on executions that avoid the corresponding C11 UB precondition.*

Lemma 8 (σ -bridge correctness). *Every σ -bridge inserted at a source/target semantic boundary preserves the recorded target value while exposing any source-side precondition violation as a candidate divergence witness.*

Lemma 9 (Variable isolation). *The product program shares only symbolic inputs; all source-local and target-local temporaries are alpha-renamed into disjoint namespaces before SMT encoding.*

References

- [1] Cybersecurity and Infrastructure Security Agency. Secure by Design pledge. <https://www.cisa.gov/securebydesign/pledge>, 2024.
- [2] National Security Agency. Software memory safety. Cybersecurity Information Sheet, November 2022.
- [3] Office of the National Cyber Director. Back to the building blocks: A path toward secure and measurable software. White House Technical Report, February 2024.
- [4] Chromium Project. Memory safety. <https://www.chromium.org/Home/chromium-security/memory-safety/>, 2019.
- [5] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. Microsoft Security Response Center, 2019.
- [6] M. Ojeda et al. Rust for Linux. <https://rust-for-linux.com/>, 2022.
- [7] P. Emre, G. Kondoh, and Z. Anderson. C2Rust: Migrating legacy code to Rust. *Proc. ICSE-Companion*, pp. 258–259, 2019.
- [8] H. Zhang, J. He, B. Ding, and Y. Zhang. Ownership guided C to Rust translation. *Proc. OOPSLA*, 2023.
- [9] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf. Translating C to safer Rust. *Proc. OOPSLA*, 2021.
- [10] ISO/IEC. ISO/IEC 9899:2011 — Programming languages — C. International Organization for Standardization, 2011.
- [11] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. *Proc. OSDI*, 2008.
- [12] Z. Rakamaric and M. Emmi. SMACK: Decoupling source language details from verifier implementations. *Proc. CAV*, 2014.
- [13] A. Gurfinkel, T. Kahsai, A. Komuravelli, and J. Navas. The SeaHorn verification framework. *Proc. CAV*, 2015.
- [14] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. Alive2: Bounded translation validation for LLVM. *Proc. PLDI*, 2021.
- [15] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. *Proc. TACAS*, pp. 151–166, 1998.
- [16] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [17] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. *Proc. TACAS*, 2008.
- [18] D. Kroening and M. Tautschnig. CBMC – C bounded model checker. *Proc. TACAS*, 2014.
- [19] Amazon Web Services. Kani Rust verifier. <https://model-checking.github.io/kani/>, 2022.

- [20] V. Astrauskas, A. Bile, P. Müller, and F. Poli. Prusti: A practical verification infrastructure for Rust. *Proc. OOPSLA*, 2022.
- [21] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.*, 2(POPL):66:1–66:34, 2018.
- [22] M. Sammler, R. Lepigre, R. Krebbers, et al. RefinedC: Automating the foundational verification of C code with refined ownership types. *Proc. PLDI*, 2021.
- [23] G. Barthe, J. M. Crespo, and C. Kunz. Relational verification using product programs. *Proc. FM*, pp. 200–214, 2011.
- [24] M. Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/>, 2024.
- [25] V. Pratt. Top down operator precedence. *Proc. POPL*, 1973.
- [26] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, 1991.
- [27] L. O. Andersen. Program analysis and specialization for the C programming language. PhD thesis, DIKU, University of Copenhagen, 1994.
- [28] J. C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.
- [29] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. *Proc. LICS*, pp. 55–74, 2002.